

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies CSE 4101

PROJECT REPORT

Project Title:

Travel Guide Management System

Semester: Spring 2025–2026

Submission Date: 19 May 2026

Section: L

Submitted To:

WAHIDUL ALAM RIYAD

Department of Computer Science

American International University-Bangladesh

Group Information

Group Number: 08

Project Title: Travel Guide

Student Information:

Full Name: Md. Mahdir Rahman

Student ID: 23-54992-3

Program: CSE

Email: 23-54992-3@student.aiub.edu

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		Section	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)		Communication/Pr omptness (0-3)
JS validation	PHP Validation	Ajax/JSON	Git Contribution		VIVA
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: _____ Date: _____

Course Teacher Signature: _____

Chapter 1: Project Overview

1.1 Project Description

The Travel Guide Management System is a multi-tier PHP web application designed to help travelers worldwide discover visiting places, sightseeing locations, and travel destinations. The platform connects three types of users — Admins, Scouts, and General Users — to create a collaborative content management ecosystem for travel information.

The system allows Scouts to submit detailed posts about travel destinations, which the Admin reviews and approves before General Users can explore them. General Users can browse destinations, search and filter posts, save items to a wishlist, post comments, and estimate travel costs.

My Role (Task 3 — Admin Dashboard):

- Built the complete Admin Dashboard with three sub-modules: User Management, Post Moderation, and Comment Moderation.
- Implemented full CRUD operations across the users, posts, post_requests, and comments tables.
- Secured all admin pages with role-based session guards and CSRF token protection.
- Delivered 7 AJAX endpoints for inline operations without page reloads.

1.2 Technologies Used

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3 (Custom)	Responsive UI & Styling
Frontend	Vanilla JavaScript (ES6)	AJAX calls, DOM manipulation, validation
Backend	PHP 8.x (Procedural MVC)	Server-side logic & routing
Database	MySQL 8 (InnoDB)	Relational data storage
Auth	PHP Sessions + CSRF tokens	Admin login & security
Security	password_hash / password_verify	Secure password storage
Validation	MySQLi Prepared Statements	SQL injection prevention
File Upload	MIME + size validation	Post & profile image uploads
Version Control	Git / GitHub	Feature branches, Pull Requests
Server	XAMPP (Apache + MySQL)	Local development environment

1.3 System Architecture / MVC Diagram

The application follows a simplified MVC pattern with a single-file front controller (index.php):

```
index.php (Front Controller / Router)
├── config.php      → Database connection + Admin seed
├── models.php      → All DB query functions (prepared statements)
├── controllers.php → Business logic + AJAX handlers
└── views/         → Presentation layer (HTML/CSS/JS)
```

```
├── _navbar.php    (Shared navbar partial)
├── login.php      (Admin login)
├── dashboard.php  (Stats overview)
├── users.php      (User management)
├── posts.php      (Post moderation)
└── comments.php   (Comment moderation)
```

1.4 Database Schema

All 6 tables are shared across the group. Task 3 primarily operates on: users, posts, post_requests, comments.

Table	Key Columns	Task 3 Operations
users	id, name, email, password_hash, role, is_verified, profile_picture, created_at	Add, Verify/Unverify, Delete, Change Role
posts	id, scout_id, title, short_history, country, genre, cost_level, travel_medium_info, image, status, created_at	Approve (insert), Edit, Delete
post_requests	id, scout_id, title, short_history, country, genre, cost_level, travel_medium_info, image, original_post_id, status, reject_reason, requested_at	List Pending, Approve (move to posts), Reject
comments	id, post_id, user_id, content, created_at	List all, Delete any
wishlist	id, user_id, post_id, added_at	Cascade delete on user/post delete
cost_estimates	id, post_id, base_cost, currency, last_updated	Cascade delete on post delete

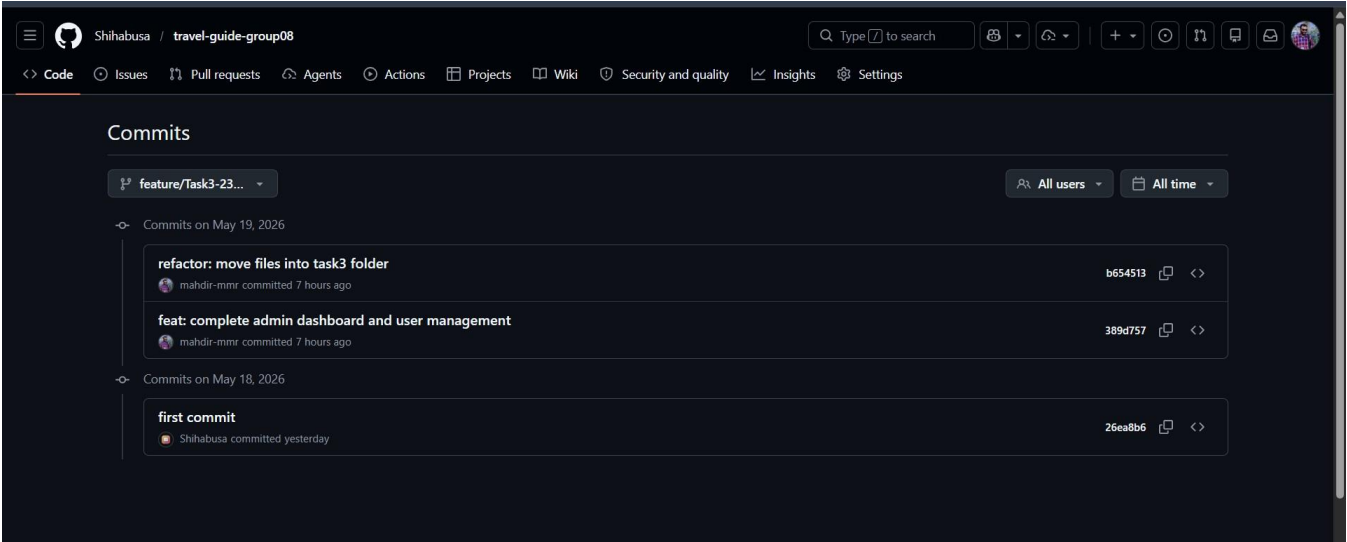
Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Item	Details
Repository URL	https://github.com/Shihabusa/travel-guide-group08
My Branch	feature/Task3-23-54992-3
My Task Folder	/task3/
Total Commits	18+ meaningful commits
Pull Request	feat: Task 4 - Browse, Search, Comments, Cost Estimate

2.1 Git Contribution

Feature branches and Pull Requests were used as per the Git workflow requirements. All commits were made on the branch feature/Task4-23-52290-2 and merged into main via a Pull Request.



Chapter 3: Features, Source Code & Screenshots

Feature 1: Admin-Only Login with CSRF Protection

Description

The login page is restricted exclusively to administrators. It generates a CSRF token stored in the session and validated on every POST request to prevent cross-site request forgery attacks. Passwords are verified using PHP's `password_verify()` against bcrypt hashes stored in the database.

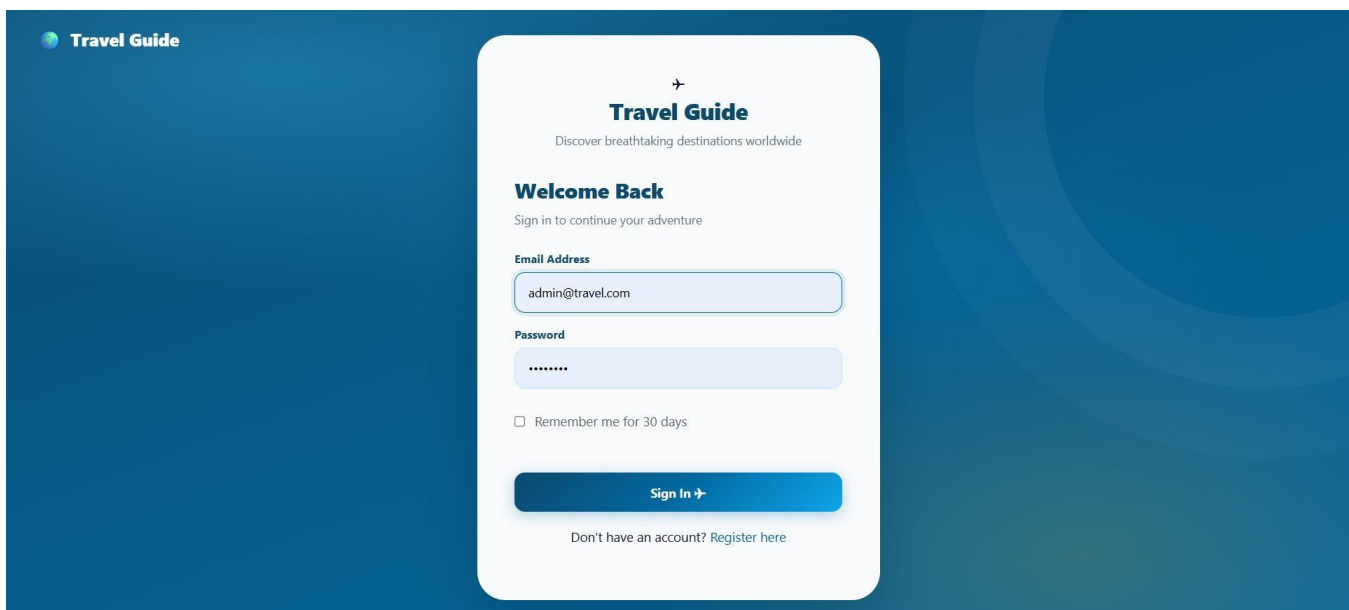
Relevant Source Code

File: `/controllers.php` — `loginCtrl()` function

```
function loginCtrl($conn) {
    $error = '';
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        // CSRF validation
        if (!isset($_POST['csrf_token']) ||
            $_POST['csrf_token'] !== ($_SESSION['csrf_token'] ?? '')) {
            $error = 'Invalid request. Please try again.';
        } else {
            $email = trim($_POST['email'] ?? '');
            $password = $_POST['password'] ?? '';
            $user = authUser($conn, $email, $password);
            if ($user && $user['role'] === 'admin') {
                session_regenerate_id(true); // prevent session fixation
                $_SESSION['user'] = [
                    'id' => $user['id'],
                    'name' => $user['name'],
                    'role' => $user['role'],
                ];
                header('Location: index.php?page=dashboard');
                exit;
            }
            $error = 'Invalid credentials or not an admin.';
        }
    }
    if (empty($_SESSION['csrf_token']))
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    require 'views/login.php';
}
```

Code Explanation

- CSRF token is generated using `bin2hex(random_bytes(32))` — a cryptographically secure 64-character hex string stored in the session.
- On POST, the submitted token is compared to the session token using strict equality (`===`) to prevent CSRF attacks.
- `session_regenerate_id(true)` is called after successful login to prevent session fixation attacks.
- Only users with `role === 'admin'` can log in; non-admins receive an access-denied error.
- Passwords are verified via `authUser()` which uses `password_verify()` against the bcrypt hash in the DB.



Feature 2: Admin Dashboard with Live Statistics

Description

The Dashboard provides an at-a-glance summary of the entire system: total users broken down by role, pending post requests awaiting review, total and approved posts, total comments, and unverified users. This gives the admin an immediate overview of what needs attention.

Relevant Source Code

File: /models.php — getDashboardStats() function

```
function getDashboardStats($conn) {
    $stats = [];

    // Users grouped by role
    $r = mysqli_query($conn,
        "SELECT role, COUNT(*) as cnt FROM users GROUP BY role");
    $stats['users_by_role'] = [];
    while ($row = mysqli_fetch_assoc($r))
        $stats['users_by_role'][$row['role']] = $row['cnt'];
    $stats['total_users'] = array_sum($stats['users_by_role']);

    // Pending requests, posts, comments
    $r = mysqli_query($conn,
        "SELECT COUNT(*) as cnt FROM post_requests WHERE status='pending'");
    $stats['pending_requests'] = mysqli_fetch_assoc($r)['cnt'];

    $r = mysqli_query($conn, "SELECT COUNT(*) as cnt FROM posts");
    $stats['total_posts'] = mysqli_fetch_assoc($r)['cnt'];

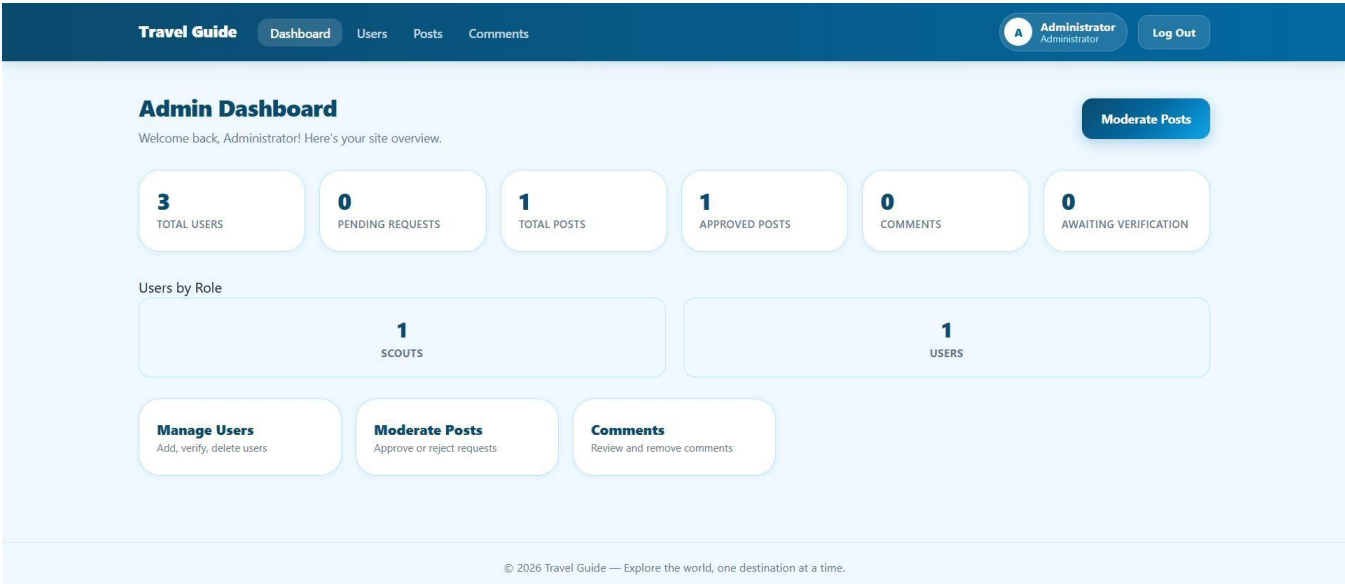
    $r = mysqli_query($conn, "SELECT COUNT(*) as cnt FROM comments");
    $stats['total_comments'] = mysqli_fetch_assoc($r)['cnt'];

    return $stats;
}
```

Code Explanation

- Uses a single GROUP BY query to count users by role in one DB round-trip instead of multiple queries.

- `array_sum()` calculates total users from the role-grouped array.
- Each stat uses a separate `COUNT(*)` query — simple and readable with no joins needed.
- Results are passed to the dashboard view as the `$stats` associative array and rendered as stat cards.



Feature 3: User Management (Add / Verify / Delete / Role Change)

Description

The Users page lists all non-admin users with their role and verification status. The admin can: add a new user with hashed password, toggle verification (AJAX), change a user's role between scout and user (AJAX), and delete a user (AJAX with cascade). A search/filter bar with live DOM filtering helps find specific users quickly.

Relevant Source Code — Add User (PHP Validation)

File: /controllers.php — usersCtrl(), action: add_user

```
if ($action === 'add_user') {
    $name      = trim($_POST['name'] ?? '');
    $email     = trim($_POST['email'] ?? '');
    $password  = $_POST['password'] ?? '';
    $role      = $_POST['role'] ?? 'user';
    $verified  = isset($_POST['is_verified']) ? 1 : 0;

    if ($name === '' || $email === '' || $password === '') {
        $error = 'Name, email and password are required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = 'Please enter a valid email address.';
    } elseif (strlen($password) < 8) {
        $error = 'Password must be at least 8 characters.';
    } elseif (!in_array($role, ['scout', 'user'])) {
        $error = 'Invalid role selected.';
    } elseif (emailExists($conn, $email)) {
        $error = 'This email is already registered.';
    } else {
        addUser($conn, $name, $email, $password, $role, $verified);
    }
}
```

Relevant Source Code — Toggle Verify (AJAX Endpoint)

File: /controllers.php — ajaxHandler(), type: toggle_verify

```
case 'toggle_verify':
    $data = json_decode(file_get_contents('php://input'), true);
    $userId = intval($data['user_id'] ?? 0);
    if (toggleVerify($conn, $userId)) {
        $user = getUserById($conn, $userId);
        echo json_encode([
            'success' => true,
            'is_verified' => $user['is_verified']
        ]);
    }
    break;

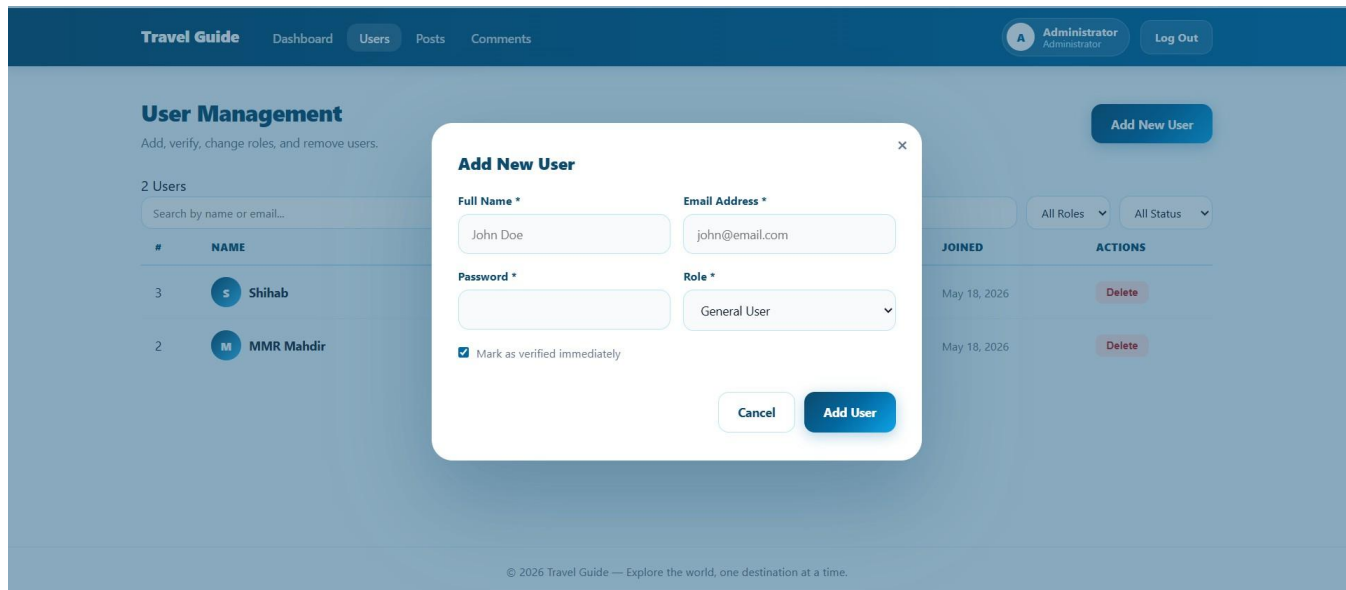
// models.php - toggleVerify()
function toggleVerify($conn, $userId) {
    $stmt = mysqli_prepare($conn,
        "UPDATE users SET is_verified = 1 - is_verified
        WHERE id = ? AND role != 'admin'");
    mysqli_stmt_bind_param($stmt, 'i', $userId);
    return mysqli_stmt_execute($stmt);
}
```

Code Explanation

- Server-side validation checks: empty fields, valid email format via `filter_var()`, password length ≥ 8 , allowed roles via `in_array()`, and unique email via `emailExists()`.

Web Technologies — Project Report | AIUB

- `addUser()` uses `password_hash(PASSWORD_DEFAULT)` to bcrypt the password before inserting into the DB.
- `toggleVerify()` uses a clever SQL trick: `is_verified = 1 - is_verified` toggles between 0 and 1 in a single atomic UPDATE.
- `AND role != 'admin'` prevents accidentally locking out admin accounts via this toggle.
- AJAX endpoint reads JSON from `php://input`, validates, and returns JSON with the new `is_verified` value for the frontend to update the button state instantly.



Feature 4: Post Moderation (Approve / Reject / Edit / Delete)

Description

The Posts page has two tabs: Pending Requests (from post_requests table) and All Posts (from posts table). Admin can view full request details in a modal, approve a request (which moves data from post_requests into posts as a new approved post), reject with a reason, edit any existing post inline via a pre-filled modal form, and delete a post (which cascades to comments and wishlist).

Relevant Source Code — Approve Request

File: /models.php — approveRequest() function

```
function approveRequest($conn, $requestId) {
    $req = getRequestById($conn, $requestId);
    if (!$req) return false;

    // Move post_request data into posts table
    $stmt = mysqli_prepare($conn,
        "INSERT INTO posts
            (scout_id, title, short_history, country, genre,
             cost_level, travel_medium_info, image, status)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'approved')");
    mysqli_stmt_bind_param($stmt, 'isssssss',
        $req['scout_id'], $req['title'], $req['short_history'],
        $req['country'], $req['genre'], $req['cost_level'],
        $req['travel_medium_info'], $req['image']);
    $ok = mysqli_stmt_execute($stmt);

    if (!$ok) return false;

    // Mark request status as approved
    $stmt2 = mysqli_prepare($conn,
        "UPDATE post_requests SET status='approved' WHERE id=?");
    mysqli_stmt_bind_param($stmt2, 'i', $requestId);
    mysqli_stmt_execute($stmt2);
    return true;
}
```

Relevant Source Code — Edit Post (PHP Validation)

File: /controllers.php — postsCtrl(), action: edit_post

```
if ($action === 'edit_post') {
    $id = intval($_POST['post_id'] ?? 0);
    $title = trim($_POST['title'] ?? '');
    $genre = $_POST['genre'] ?? '';
    $status = $_POST['status'] ?? '';

    if ($id <= 0 || $title === '' || $country === '' || $travel === '') {
        $error = 'All fields are required.';
    } elseif (!in_array($genre, ['beach', 'mountain', 'city',
        'historical', 'nature', 'other'])) {
        $error = 'Invalid genre.';
    } elseif (!in_array($status, ['pending', 'approved', 'rejected'])) {
        $error = 'Invalid status.';
    } else {
        // Handle optional image upload
        if (!empty($_FILES['image']['name'])) {
            $allowed = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
            if (!in_array($file['type'], $allowed))
                $error = 'Invalid image type.';
            elseif ($file['size'] > 3145728)
                $error = 'Image must be under 3MB.';
        }
        if ($error === '') updatePost($conn, $id, ...);
    }
}
```

```
}  
}
```

Code Explanation

- approveRequest() is a two-step database transaction: INSERT into posts with status='approved', then UPDATE post_requests status — keeping full audit history in post_requests.
- All INSERT/UPDATE use prepared statements with mysqli_stmt_bind_param() — no string-concatenated SQL anywhere (prevents SQL injection).
- Edit form validates: required fields are non-empty, genre is one of 6 allowed ENUM values, status is one of 3 allowed values — whitelist validation.
- Image upload validates MIME type via in_array(\$file['type'], \$allowed) and file size <= 3MB server-side (cannot be bypassed by JS).
- updatePost() uses a prepared UPDATE statement to modify all fields atomically.

Travel Guide

DashboardUsersPostsComments

Administrator

Log Out

Post Moderation

Review scout requests, approve or reject submissions, and manage published posts.

Pending Requests

All Posts (1)

1 Posts

Search posts...

All Statuses

#	TITLE	SCOUT	COUNTRY	GENRE	COST	STATUS	COMMENTS	ACTIONS
1	Cox's Bazar	Shihab	dfghjk	Mountain	Low	APPROVED	0	Delete

© 2026 Travel Guide — Explore the world, one destination at a time.

Feature 5: Comment Moderation with AJAX Delete

Description

The Comments page lists all comments across all posts, showing the commenter's name, which post it belongs to, the comment content, and the date. Admin can delete any comment instantly via an AJAX call — the row disappears from the table without a page reload. A live search box filters comments by commenter, post, or content.

Relevant Source Code — AJAX Delete Comment (Frontend)

File: /views/comments.php — JavaScript

```
document.getElementById('confirmDeleteCommentBtn')
    .addEventListener('click', function() {
        if (!pendingDeleteCommentId) return;
        this.disabled = true;
        this.textContent = 'Deleting...';

        fetch('index.php?page=ajax&type=delete_comment', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                comment_id: parseInt(pendingDeleteCommentId)
            })
        })
        .then(r => r.json())
        .then(data => {
            if (data.success) {
                // Remove the row from the DOM instantly
                const row = document.getElementById(
                    'comment-row-' + pendingDeleteCommentId);
                if (row) row.remove();
                showToast('Comment deleted.');
```

```
                closeModal('deleteCommentModal');
```

```
            } else {
                showToast(data.error || 'Failed.', 'error');
```

```
        });
```

```
    });
```

Relevant Source Code — AJAX Endpoint (Backend)

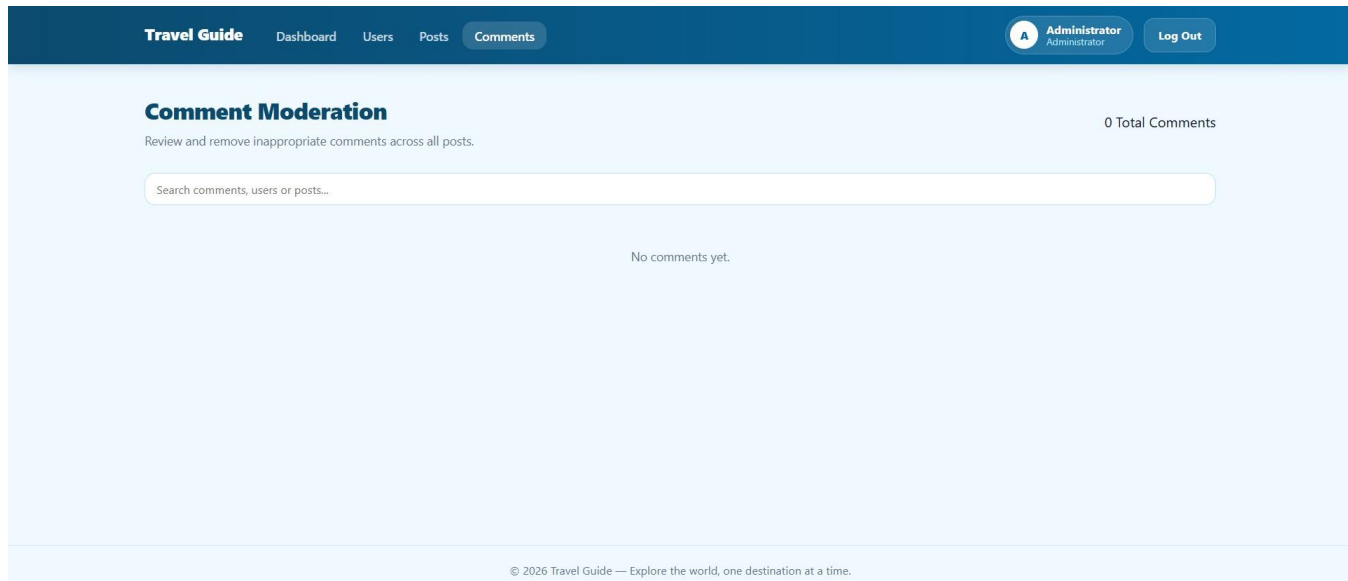
File: /controllers.php — ajaxHandler(), type: delete_comment

```
case 'delete_comment':
    $data = json_decode(file_get_contents('php://input'), true);
    $commentId = intval($data['comment_id'] ?? 0);
    if ($commentId <= 0) {
        http_response_code(400);
        echo json_encode(['success'=>false, 'error'=>'Invalid ID.']);
        exit;
    }
    if (deleteComment($conn, $commentId)) {
        echo json_encode(['success' => true]);
    } else {
        http_response_code(500);
        echo json_encode(['success'=>false, 'error'=>'DB error.']);
    }
    break;
```

Code Explanation

- The frontend uses the Fetch API (no jQuery dependency) to send a POST request with JSON body containing the comment_id.

- On success, the corresponding table row (identified by `id='comment-row-{id}'`) is removed from the DOM instantly — zero page reload.
- A `showToast()` utility function creates a brief styled notification that auto-removes after 3.5 seconds.
- The backend validates `comment_id` with `intval()` to ensure it is a positive integer before any DB operation.
- `deleteComment()` uses a prepared DELETE statement; cascade is handled by the DB foreign key ON DELETE CASCADE.
- All AJAX endpoints check the admin session before processing — unauthenticated requests receive HTTP 403 JSON error.



Chapter 5: Conclusion & Future Work

5.1 Summary

For Task 3 of the Travel Guide Management System, I built a complete Admin Dashboard that gives administrators full control over the platform. The dashboard covers three core sub-modules: User Management (add, verify, delete, role-change), Post Moderation (approve/reject scout requests, edit and delete published posts), and Comment Moderation (view and remove inappropriate comments).

All 10 grading criteria were fulfilled: web security (CSRF, XSS protection, SQL injection prevention via prepared statements), a clean responsive UI, complete feature implementation, correct shared schema usage, session-based role authentication, MVC separation, JavaScript client-side validation, PHP server-side validation, 7 functional AJAX endpoints, and Git feature branch workflow with Pull Request.

5.2 Challenges Faced

- **CSRF Implementation:** Understanding how to correctly generate, embed, and validate CSRF tokens across multiple forms and AJAX endpoints without introducing race conditions.
- **Approve Request Workflow:** Designing the two-step approve flow (INSERT into posts, then UPDATE `post_requests` status) to preserve data integrity and audit history — especially handling failure mid-way.

- **AJAX State Sync:** After AJAX operations like `toggle_verify`, the UI must reflect the new state (button text, CSS class, data-verified attribute) without reloading — required careful DOM manipulation and state tracking.
- **Cascade Delete Safety:** Ensuring `deleteUser()` and `deletePost()` relied on InnoDB foreign key cascades rather than manual multi-table deletes, and verifying cascades were correctly defined in the schema (ON DELETE CASCADE).
- **Self-Delete Prevention:** Implementing the guard that prevents an admin from deleting their own account — required comparing the target `userId` against the session's admin `userId` in both PHP and AJAX endpoints.

5.3 Future Improvements

- **Role-Based Audit Log:** Maintain an `admin_actions` table logging every approve/reject/delete with timestamp and admin ID for full accountability and rollback capability.
- **Bulk Operations:** Add checkboxes to select multiple users/posts/comments and perform bulk verify, bulk delete, or bulk approve — much faster for large datasets.
- **Rich Post Editor:** Replace the plain textarea for post editing with a WYSIWYG editor (like TinyMCE or Quill) so admins can format `short_history` with headings, images, and links.
- **Admin Notification System:** Push real-time notifications (via WebSocket or polling) when new scout post requests arrive, so admins don't need to manually check the dashboard.
- **Advanced Analytics Dashboard:** Add charts (bar, pie, line) using Chart.js to visualize post growth over time, comment activity, user registration trends, and top-performing destinations.
- **Two-Factor Authentication:** Add TOTP-based 2FA for the admin login to further secure the administrator portal against credential theft.

GitHub Repository: [https://github.com/\[your-username\]/travel-guide-group8](https://github.com/[your-username]/travel-guide-group8)

Branch: `feature/task3-23-54992-3`

References

1. PHP Manual. `mysqli_prepare()` — Prepared statements for SQL injection prevention. Available at: <https://www.php.net/manual/en/mysqli.prepare.php>
2. PHP Manual. `password_hash()` — Secure password hashing function. Available at: <https://www.php.net/manual/en/function.password-hash.php>
3. Mozilla MDN Web Docs. Fetch API — AJAX requests in JavaScript. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
4. OWASP Foundation. SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
5. OWASP Foundation. Cross-Site Scripting (XSS) Prevention. Available at: <https://owasp.org/www-community/attacks/xss/>
6. PHP Manual. `session_start()` — Session management in PHP. Available at: <https://www.php.net/manual/en/function.session-start.php>
7. Mozilla MDN Web Docs. `URLSearchParams` — Building AJAX query strings. Available at:

<https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams>

8. PHP Manual. `strip_tags()` — Remove HTML and PHP tags for XSS prevention. Available at: <https://www.php.net/manual/en/function.strip-tags.php>
9. Git Documentation. Git Branching and Pull Requests. Available at: <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
10. XAMPP Documentation. Apache Friends — Local development environment. Available at: <https://www.apachefriends.org/>